

CS50 — Semaine 7

SQL & bases de donnees relationnelles

Fiche exhaustive — Doug Lloyd shorts, lecture Malan, section Kelly

Pour Flavien — preparation Phase 1, transition AI/ML.

Code couleur : **violet** = Malan, **orange** = Doug Lloyd, **vert** = Kelly, **bleu** = analogies, **rouge** = concepts cles, monospace = code.

Comment utiliser cette fiche. Lecture lineaire la premiere fois pour construire le modele mental. Puis Ctrl+F pendant les PSets : tu cherches JOIN, GROUP BY, ? placeholders, et tu retombes sur la section qui explique le concept avec son piege.

TL;DR de la semaine

SQL (Structured Query Language) est un langage **declaratif** : tu écris ce que tu veux, le moteur de base de données trouve comment l'obtenir. C'est fondamentalement différent de C ou Python où tu écris des boucles et des conditions. Toute la semaine tient en quatre verbes : INSERT, SELECT, UPDATE, DELETE (acronyme CRUD).

Les données vivent dans des **tables** (lignes + colonnes typées). Chaque ligne possède une **cle primaire** qui l'identifie de manière unique. Quand une cle d'une table apparaît dans une autre table, on l'appelle **cle étrangère** — c'est ce mécanisme qui rend la base **relationnelle**.

Trois types de relations entre tables : **1-1** (un film a une note moyenne), **1-N** (un film a plusieurs genres), **N-N** (plusieurs acteurs jouent dans plusieurs films). Pour interroger plusieurs tables ensemble : **nested SELECT** (lisible, conseille pour débuter) ou **JOIN** (puissant mais plus abstrait).

Trois pièges du monde réel à connaître maintenant car ils reviendront en production : **SQL injection** (jamais de f-strings pour le user input, toujours ? placeholders), **race conditions** (deux serveurs qui modifient la même valeur en même temps perdent des données, solution = transactions), **indexes** (sans index, recherche linéaire ; avec CREATE INDEX sur un B-tree, recherche logarithmique — ordres de grandeur plus rapide).

CONCEPT CLE

La phrase à retenir : SQL n'est pas juste un nouveau langage, c'est un nouveau *paradigme*. Tu passes de procédural (je dis comment) à déclaratif (je dis quoi). C'est ce changement de posture qui rend SQL court et puissant : une seule ligne SQL peut remplacer 20 lignes de Python.

1. Declaratif vs procedural — le changement de paradigme

ANALOGIE

Magny-Cours. En procedural (C, Python), tu es ingénieur de course : tu expliques chaque virage, chaque rapport de boîte, chaque freinage. En déclaratif (SQL), tu es directeur sportif : tu dis *amène-moi en pole*, et le pilote (le moteur SQL) trouve la trajectoire. L'avantage : ton instruction tient en une phrase. L'inconvénient : tu dois faire confiance au pilote et tu ne contrôles pas comment il s'y prend.

Malan illustre la différence avec un fichier CSV de favoris récoltés pendant le cours. La même question (*combien d'élèves préfèrent Python ?*) demande environ 20 lignes en Python (ouvrir le fichier, parser les lignes, dictionnaire, boucle, incrément, gestion d'erreurs) et **une seule ligne** en SQL :

```
SELECT language, COUNT(*) AS n
FROM favorites
GROUP BY language
ORDER BY n DESC;
```

DAVID MALAN — lecture

« SQL is said to be a declarative programming language [...] when you want to solve some problem you essentially declare what problem you want to solve [...] and it's up to the programming language to figure out using loops and conditionals and all of those lower level building blocks, how to get you the answer. »

Implication pour toi : arrête de penser en termes de *boucle qui parcourt*. Pense en termes de *filtre* + *agregation*. La requête décrit l'**ensemble résultat** que tu veux, pas les étapes pour l'obtenir.

2. Vocabulaire — database, table, ligne, colonne

DOUG LLOYD — shorts

Doug Lloyd : analogie Excel. Un fichier Excel Book1 contient plusieurs *sheets* ; chaque sheet a des colonnes (A, B, C...) et des lignes (1, 2, 3...). En SQL : **database** = fichier, **table** = sheet, **colonne** = colonne typée, **ligne** = enregistrement (un client, une chanson, un user).

Hierarchie : une **database** contient une ou plusieurs **tables**. Chaque table est définie par un **schema** (la liste de ses colonnes et de leurs types). Les données sont stockées ligne par ligne. Tu interroges la table avec des requêtes SQL qui te renvoient un sous-ensemble de lignes/colonnes.

SQLite vs MySQL vs Postgres vs Oracle. Ce sont tous des *moteurs* qui implémentent SQL. SQLite est lightweight (un seul fichier .db, pas de serveur, pas de mots de passe), c'est ce que

CS50 utilise et c'est très courant en apps mobiles et web embarquées. MySQL/Postgres/Oracle sont des serveurs centraux avec gestion d'utilisateurs, permissions, scaling — ce que tu retrouveras en entreprise.

CONCEPT CLE

Pour CS50 : `sqlite3 nom.db` dans le terminal. C'est tout. Le 3 est juste le numéro de version, pas un détail à mémoriser. Tout ce qui commence par `.` (point) dans le prompt SQLite est spécifique à SQLite (ex: `.schema`, `.import`, `.mode`). Tout le reste est du SQL standard portable.

3. Types de données — SQLite vs SQL standard

En C, tu connaissais `int`, `float`, `char`, `string`. SQL standard ajoute beaucoup plus de types (`date`, `time`, `geometry`, `blob`, `enum`, `varchar` de longueur variable...). SQLite simplifie tout cela en 5 **affinités de type** :

Affinité SQLite	Équivalent C / Python	Exemples d'usage
INTEGER	<code>int</code> (1, 2, 3, ...)	Cles primaires, âge, compteurs
REAL	<code>float</code> / <code>double</code>	Prix, ratings, mesures
TEXT	<code>string</code>	Noms, titres, descriptions
BLOB	<code>bytes</code> (raw)	Images, fichiers binaires (rare)
NULL	<code>None</code> / <code>NULL</code>	Absence consciente de valeur

Détail important : NULL. Ce n'est pas une chaîne vide, c'est l'*absence consciente* de donnée. C'est l'équivalent de `None` en Python ou `NULL` en C (le pointeur). Quand tu cherches des lignes où un champ est manquant, tu n'utilises pas `= NULL` (qui ne marche pas) mais `IS NULL` ou `IS NOT NULL`.

```
-- Faux : ne renvoie rien même si time_stamp est NULL
DELETE FROM favorites WHERE time_stamp = NULL;

-- Vrai : utilise le prédicat IS NULL
DELETE FROM favorites WHERE time_stamp IS NULL;
```

DOUG LLOYD — shorts

CHAR(n) vs VARCHAR(n). Pas vu en SQLite mais important pour MySQL/Postgres que tu rencontreras. `CHAR(10)` = chaîne de longueur *fixe* à 10 caractères : si tu stockes «hi», ça prend quand même 10 octets (8 octets vides ajoutés). `VARCHAR(99)` = longueur *variable* jusqu'à 99 max. `VARCHAR` économise de l'espace mais `CHAR` est plus rapide à indexer (longueur prédictible).

4. Cle primaire & autoincrement

Toute table devrait avoir une **cle primaire** : une colonne (ou combinaison de colonnes) qui identifie de maniere *unique* chaque ligne. Sans cle primaire, tu peux avoir des doublons impossibles a distinguer ; avec elle, tu peux retrouver et referencer une ligne en $O(\log n)$ si la base est indexee.

ANALOGIE

Optic 2000. Ton client «Marie Dupont» n'est pas identifie par son nom (il y a plusieurs Marie Dupont) mais par son **numero de dossier**. C'est exactement ca une cle primaire. Et quand tu emets une facture, tu y mets ce numero de dossier — pas le nom du client. Si demain Marie Dupont change de nom de famille, ton numero ne bouge pas, donc rien ne casse.

Convention forte : la cle primaire est typiquement une colonne nommee `id` de type `INTEGER` avec *autoincrement*. SQLite gere ca automatiquement — si tu fais un `INSERT` sans specifier l'`id`, SQLite genere le prochain entier libre.

```
CREATE TABLE friend (  
    id INTEGER PRIMARY KEY,  
    name TEXT NOT NULL,  
    age INTEGER NOT NULL  
);  
  
-- INSERT sans id : SQLite met 1 automatiquement  
INSERT INTO friend (name, age) VALUES ('Alice', 25);  
-- Puis 2, puis 3, ... a chaque nouvel INSERT
```

CONCEPT CLE

Cle primaire composite (joint primary key). Parfois aucune colonne seule n'est unique, mais la *combinaison* de deux colonnes l'est. Exemple : table `stars` qui lie un film a un acteur. Ni `movie_id` ni `person_id` n'est unique seul (un acteur joue dans plusieurs films, un film a plusieurs acteurs), mais le couple (`movie_id`, `person_id`) doit l'etre — on ne veut pas dire deux fois que Steve Carell joue dans The Office.

Cle etrangere (foreign key). Quand la cle primaire d'une table apparait comme colonne dans une autre table, on l'appelle cle *etrangere* dans la seconde. C'est le mecanisme qui cree les *relations*. Convention de nommage courante : table `shows` a une colonne `id` (primaire), table `ratings` a une colonne `show_id` (etrangere) qui pointe vers `shows.id`.

```
CREATE TABLE ratings (  
    show_id INTEGER NOT NULL,  
    rating REAL NOT NULL,  
    votes INTEGER NOT NULL,  
    FOREIGN KEY(show_id) REFERENCES shows(id)  
);
```

5. CRUD — INSERT (créer une ligne)

Syntaxe générale :

```
INSERT INTO table_name (col1, col2, ...)
VALUES (val1, val2, ...);
```

Tu specifies les colonnes que tu remplis (dans l'ordre que tu veux), puis les valeurs correspondantes (dans le même ordre). Les colonnes que tu omets prennent leur valeur par défaut, ou NULL, ou s'auto-incrementent (cas de la clé primaire).

```
-- Insertion complète
INSERT INTO users (username, password, fullname)
VALUES ('newman', 'USMAIL', 'Newman');

-- L'id est généré automatiquement (12, 13, 14...)
```

DOUG LLOYD — shorts

Doug montre un piège classique : si tu définis ta clé primaire en INTEGER mais que tu oublies de la mettre en auto-increment et que tu ne la fournis pas dans tes INSERT, tu finis avec plusieurs lignes à NULL sur l'id — et donc non identifiables uniquement. C'est pour ça que la combinaison INTEGER PRIMARY KEY en SQLite (qui auto-increment implicitement) est la bonne pratique par défaut.

6. CRUD — SELECT (lire des données)

C'est la requête que tu utiliseras 95% du temps. Syntaxe générale :

```
SELECT colonne1, colonne2, ...
FROM table_name
[WHERE condition]
[GROUP BY colonne]
[ORDER BY colonne ASC|DESC]
[LIMIT n];
```

Les crochets [] indiquent que la clause est optionnelle. L'ordre est **impératif** : WHERE avant GROUP BY avant ORDER BY avant LIMIT. Sinon erreur de syntaxe.

SELECT * — toutes les colonnes

```
SELECT * FROM songs;
-- L'asterisque est un wildcard : 'donne-moi toutes les colonnes'
-- Pratique en exploration, jamais en production (trop de données)
```

WHERE — filtrer les lignes

```

SELECT name FROM songs WHERE tempo > 100;

-- Operateurs : = != < > <= >= AND OR NOT
SELECT * FROM users WHERE id < 12 AND fullname != 'Newman';

-- IN : appartenance a une liste
SELECT * FROM songs WHERE artist_id IN (1, 5, 12);

-- BETWEEN : intervalle inclusif
SELECT * FROM songs WHERE tempo BETWEEN 100 AND 140;

```

LIKE — pattern matching de chaines

Le caractere % est un wildcard qui matche zero ou plusieurs caracteres. Le caractere _ matche exactement un caractere. **Different de l'asterisque * qui veut dire «toutes les colonnes».**

```

-- Tous les noms qui commencent par 'hello'
SELECT name FROM songs WHERE name LIKE 'hello%';

-- Tous les noms qui contiennent 'love'
SELECT name FROM songs WHERE name LIKE '%love%';

-- Tous les codes course CS suivis de exactement 2 caracteres
SELECT code FROM courses WHERE code LIKE 'CS__';

-- Astuce : LIKE est case-insensitive en SQLite (love == LOVE)

```

ORDER BY — trier le resultat

```

-- Trier par tempo croissant (ASC est le default)
SELECT name, tempo FROM songs ORDER BY tempo;
SELECT name, tempo FROM songs ORDER BY tempo ASC; -- pareil

-- Decroissant
SELECT name, tempo FROM songs ORDER BY tempo DESC;

-- Tri sur plusieurs colonnes
SELECT * FROM songs ORDER BY artist_id, tempo DESC;

```

LIMIT — limiter le nombre de lignes

```

-- Top 5 chansons les plus longues
SELECT name FROM songs
ORDER BY duration_ms DESC
LIMIT 5;

-- LIMIT s'applique APRES le tri.
-- Sans ORDER BY, l'ordre des resultats n'est pas garanti.

```

Alias avec AS — renommer une colonne du resultat

```
-- Sans alias : la colonne s'appelle 'COUNT(*)' (moche)
SELECT language, COUNT(*) FROM favorites GROUP BY language;

-- Avec alias : on l'appelle n, plus court a reutiliser
SELECT language, COUNT(*) AS n
FROM favorites
GROUP BY language
ORDER BY n DESC;
```


7. Fonctions d'agregation — COUNT, AVG, MIN, MAX, SUM, DISTINCT

Les fonctions d'agregation reduisent un ensemble de lignes en **une seule valeur**. Au lieu de te renvoyer 100 lignes, elles te renvoient un nombre.

Fonction	Ce qu'elle fait	Exemple
COUNT(*)	Compte le nb de lignes	SELECT COUNT(*) FROM songs;
COUNT(col)	Compte les valeurs non-NULL de col	SELECT COUNT(rating) FROM ratings;
AVG(col)	Moyenne arithmetique	SELECT AVG(energy) FROM songs;
MIN(col)	Plus petite valeur	SELECT MIN(year) FROM shows;
MAX(col)	Plus grande valeur	SELECT MAX(votes) FROM ratings;
SUM(col)	Somme	SELECT SUM(epsodes) FROM shows;
DISTINCT col	Valeurs uniques (sans doublons)	SELECT DISTINCT language FROM favorites;

GROUP BY — agreger par categorie

Sans GROUP BY, une agregation comme COUNT(*) renvoie une seule ligne. Avec GROUP BY col, elle renvoie une ligne par valeur distincte de col. C'est l'equivalent de *regrouper les lignes identiques sur cette colonne et compter chaque groupe*.

```
-- Combien d'eleves ont choisi chaque langage ?
SELECT language, COUNT(*) AS n
FROM favorites
GROUP BY language
ORDER BY n DESC;

-- Resultat :
-- language | n
-- Python   | 190
-- C        | 58
-- Scratch  | 24
```

KELLY — section

Kelly explique en section : GROUP BY est une operation mentale en deux temps. Etape 1 : SQL regroupe toutes les lignes qui ont la meme valeur sur la colonne mentionnee. Etape 2 : pour chaque groupe, il applique la fonction d'agregation (COUNT, AVG...) et renvoie une seule ligne par groupe. Si tu visualises cette etape de regroupement, GROUP BY n'est plus magique.

8. CRUD — UPDATE et DELETE (modifier, supprimer)

UPDATE — modifier des lignes existantes :

```
UPDATE table_name
SET colonne = nouvelle_valeur
WHERE condition;

-- Exemple : Alice a 26 ans maintenant
UPDATE friend SET age = 26 WHERE name = 'Alice';

-- Plusieurs colonnes en meme temps
UPDATE users SET password = 'yadayada', email = 'new@x.com'
WHERE id = 10;
```

DELETE — supprimer des lignes :

```
DELETE FROM table_name WHERE condition;

-- Exemple : retirer Bob de mes amis
DELETE FROM friend WHERE name = 'Bob';

-- Supprimer toutes les lignes orphelines (sans timestamp)
DELETE FROM favorites WHERE time_stamp IS NULL;
```

CONCEPT CLE

LE PIEGE LE PLUS DANGEREUX DE TOUT SQL. Si tu oublies WHERE :

- UPDATE friend SET age = 26; → *tous* tes amis ont 26 ans.
- DELETE FROM friend; → *tous* tes amis sont effaces.

Malan raconte des cas reels d'internes en entreprise qui ont fait ca en production. Toujours, toujours, toujours verifie ton WHERE avant d'appuyer Entree. Reflexe : ecris d'abord SELECT * FROM table WHERE condition pour voir *quelles* lignes seront affectees, puis remplace le SELECT par UPDATE/DELETE.

DROP TABLE — encore plus dangereux :

```
DROP TABLE friend;
-- Supprime la table ENTIERE (donnees + structure).
-- Pas de WHERE, pas de retour en arriere sans backup.
```

9. Relations — le coeur du «relationnel»

Une base de données est dite **relationnelle** parce qu'elle te permet de définir des *relations* entre tables au lieu de tout entasser dans une seule grosse table. Trois types de relations existent :

Type	Definition	Exemple IMDB
1-1	Une ligne A correspond a exactement une ligne B	Un show a une seule entree de rating
1-N	Une ligne A peut avoir plusieurs lignes B associees	Un show a plusieurs genres
N-N	Plusieurs A correspondent a plusieurs B (et inversement)	Plusieurs acteurs dans plusieurs shows

ANALOGIE

Optic 2000. Trois exemples concrets pris dans ton metier :

- **1-1** : un client a un seul dossier securite sociale (1 client ↔ 1 numero secu).
- **1-N** : un client a achete plusieurs paires de lunettes au cours du temps.
- **N-N** : plusieurs clients ont commande chez plusieurs verriers (Essilor, Zeiss, etc.). Pour modeliser ce N-N proprement, tu crees une table de jonction commandes avec deux cles etrangeres : `client_id` et `verrier_id`.

Le piege que tu DOIS éviter : tout mettre dans une seule table

Malan montre la mauvaise approche dans une feuille Excel : pour stocker une serie TV avec ses acteurs, tu ajoutes des colonnes `star1`, `star2`, `star3`, `star4`... Probleme : tu ne sais pas combien de colonnes prevoir, beaucoup de lignes auront des cellules vides, c'est ingerable.

Deuxieme tentative : une seule table avec une ligne par couple (show, star). Probleme : tu repetes le titre du show 50 fois, gaspillage et risque de fautes de frappe.

Bonne approche — trois tables :

```
shows   : id, title, year, episodes
people  : id, name, birth_year
stars   : show_id, person_id (table de jonction N-N)
```

C'est ce qu'on appelle **normaliser** les données : eliminer toutes les redondances en separant les entites distinctes dans leurs propres tables, et lier les tables avec des cles. Resultat : pas de doublons textuels, juste des ids entiers (de longueur fixe = rapides a indexer).

10. Nested SELECT — requetes imbriquees

Premiere facon d'interroger plusieurs tables : **imbriquer** les requetes. Tu fais un SELECT a l'interieur du WHERE d'un autre SELECT. SQL execute la requete *interne* d'abord, puis utilise son resultat dans la requete *externe*.

Exemple : trouver tous les genres du show 'Catweazle'

Probleme : la table shows a le titre, la table genres a (show_id, genre). Comment lier les deux sans connaitre l'id de Catweazle a l'avance ?

```
-- Etape mentale 1 : on a besoin de l'id de Catweazle
SELECT id FROM shows WHERE title = 'Catweazle';
-- Renvoie : 63881

-- Etape mentale 2 : on l'utilise pour filtrer les genres
SELECT genre FROM genres WHERE show_id = 63881;
-- Renvoie : Adventure / Comedy / Family

-- En une seule requete imbriquee :
SELECT genre FROM genres
WHERE show_id = (
    SELECT id FROM shows WHERE title = 'Catweazle'
);
```

= vs IN — choix critique

Si la requete interne renvoie **une seule valeur**, utilise =. Si elle renvoie **plusieurs valeurs** (une colonne entiere), utilise IN.

```
-- Show ID unique : on utilise =
SELECT genre FROM genres
WHERE show_id = (SELECT id FROM shows WHERE title = 'Catweazle');

-- Plusieurs ids possibles : on utilise IN
SELECT title FROM shows
WHERE id IN (
    SELECT show_id FROM ratings WHERE rating >= 6.0
);
```

KELLY — section

Kelly recommande explicitement les **nested SELECT pour debuter**. C'est plus lisible parce que tu progresses du probleme interieur vers l'exterieur, etape par etape : d'abord trouver l'id, puis utiliser cet id pour filtrer. JOIN est plus puissant mais demande de penser a la structure entiere d'un coup — plus difficile au debut.

11. JOIN — fusionner deux tables temporairement

Deuxieme facon : **JOIN**. Au lieu d'imbriquer, tu fusionnes deux tables en une table virtuelle temporaire en alignant les lignes via leurs cles communes. Le resultat ressemble a une feuille Excel ou les deux tables seraient collees cote a cote.

Syntaxe :

```
SELECT col1, col2
FROM table_a
JOIN table_b ON table_a.cle = table_b.cle_etrangere
WHERE condition;
```

Exemple 1-1 : titre + rating

```
SELECT title, rating
FROM shows
JOIN ratings ON shows.id = ratings.show_id
WHERE rating >= 6.0
LIMIT 10;
```

Pour chaque ligne de shows, SQL cherche la ligne correspondante dans ratings (celle ou show_id egale l'id du show). Il fusionne les deux lignes, puis applique le filtre WHERE.

Exemple N-N : tous les acteurs de The Office

Tu dois passer par 3 tables : shows, stars (table de jonction), people. Deux JOIN successifs :

```
SELECT name FROM people
JOIN stars ON people.id = stars.person_id
JOIN shows ON stars.show_id = shows.id
WHERE shows.title = 'The Office'
      AND shows.year = 2005;
```

Notation table.colonne obligatoire des qu'une colonne porte le meme nom dans plusieurs tables (ici id existe dans people ET dans shows).

Meme question, version nested SELECT

```
SELECT name FROM people
WHERE id IN (
    SELECT person_id FROM stars
    WHERE show_id = (
        SELECT id FROM shows
        WHERE title = 'The Office' AND year = 2005
    )
);
```

CONCEPT CLE

JOIN ou nested SELECT, lequel choisir ?

- **Nested SELECT** : plus lisible quand tu apprends. Tu lis l'imbrication de l'interieur vers l'exterieur. Souvent plus rapide pour des cas simples (SQL execute la requete interne une fois et reutilise le resultat).
- **JOIN** : plus puissant quand tu as besoin des donnees de plusieurs tables dans le resultat. Indispensable des que tu veux afficher cote a cote des infos venant de tables differentes (titre du show + nom de l'acteur, par exemple). Souvent plus rapide quand les indexes sont bien places.

Pour le PSet 7 : commence en nested. Tu passeras a JOIN quand le besoin se presentera (typiquement Movies).

12. Indexes — rendre les requetes radicalement plus rapides

Par default, quand tu fais `SELECT * FROM shows WHERE title = 'The Office'`, SQL fait une **recherche lineaire** : il scanne la table de haut en bas en comparant chaque ligne. Sur 250 000 lignes, c'est lent. C'est le meme probleme que tu as resolu avec la recherche binaire semaine 5 de CS50.

ANALOGIE

Index telephonique a l'ancienne. Sans index, pour trouver «Dupont Marie», tu dois lire chaque page du bottin de la premiere a la derniere. Un index trie alphabetiquement te permet d'aller directement a la lettre D, puis a Du, puis a Dupont — en quelques sauts au lieu de milliers de lignes. C'est exactement ce que fait `CREATE INDEX` : il construit en memoire un arbre (B-tree) qui te permet d'atteindre la donnee en $O(\log n)$ au lieu de $O(n)$.

Syntaxe :

```
CREATE INDEX nom_index ON table (colonne);

-- Exemple : indexer la colonne title de shows
CREATE INDEX title_index ON shows (title);
```

Demonstration de Malan :

```
.timer ON -- active le chronometrage des requetes

-- Avant index :
SELECT * FROM shows WHERE title = 'The Office';
-- Real time : 0.042 seconds

CREATE INDEX title_index ON shows (title);

-- Apres index, MEME requete :
SELECT * FROM shows WHERE title = 'The Office';
-- Real time : 0.001 seconds (40x plus rapide)
```

CONCEPT CLE

Trade-off : les indexes accelerent les `SELECT` mais ralentissent les `INSERT`, `UPDATE` et `DELETE` (parce que SQL doit aussi mettre a jour l'arbre). Sur des tables tres mises a jour, n'indexe que les colonnes que tu interrogues frequemment. Pour le PSet, indexer une ou deux colonnes peut faire passer une requete de minutes a millisecondes — observe ca avec `.timer ON`.

13. SQL Injection — le piege de securite N°1

Quand tu integres SQL dans un autre langage (Python, PHP...), tu vas vouloir construire des requetes a partir de l'input utilisateur. C'est la que le pire bug de l'histoire du web survient.

La mauvaise facon — ne fais JAMAIS ca :

```
favorite = input('Ton probleme prefere : ')
rows = db.execute(
    f"SELECT COUNT(*) FROM favorites WHERE problem = '{favorite}'"
)
```

Que se passe-t-il si l'utilisateur tape hello, it's me ? L'apostrophe ferme prematurement la chaine et SQLite hurle. Pire : si l'utilisateur est malveillant et tape ' ; DROP TABLE favorites ; -- , ta requete devient :

```
SELECT COUNT(*) FROM favorites WHERE problem = '';
DROP TABLE favorites; --';
-- Le -- est un commentaire SQL qui ignore tout le reste.
```

Resultat : ta table est detruite. C'est ca une **SQL injection**.

La bonne facon — toujours des placeholders :

```
from cs50 import SQL
db = SQL('sqlite:///favorites.db')

favorite = input('Ton probleme prefere : ')
rows = db.execute(
    'SELECT COUNT(*) AS n FROM favorites WHERE problem = ?',
    favorite
)
```

Le ? est un **placeholder**. La librairie (CS50, sqlite3, pycopg2...) recoit la valeur a part et l'*echappe* proprement avant de l'insérer. Tous les caracteres dangereux (apostrophes, point-virgules, backslashes) sont neutralises automatiquement.

DAVID MALAN — lecture

Malan termine la lecture sur le meme exemple : si GitHub etait vulnerable a ce bug, taper malan@harvard.edu -- dans le champ username permettrait a n'importe qui de se connecter en tant que Malan sans connaitre son mot de passe. La librairie qui echappe l'apostrophe transforme ca en une simple chaine bizarre qui ne matche aucun utilisateur reel.

CONCEPT CLE

Regle absolue. Jamais, jamais, jamais de f-string ou de concatenation Python pour construire du SQL avec de l'input utilisateur. Toujours des ? et la librairie qui echappe pour toi. C'est une regle qu'on n'enfreint pas, meme «juste pour tester».

14. Race conditions — quand deux serveurs s'ecrasent

ANALOGIE

Le frigo a colocs. Tu rentres, frigo vide, tu pars acheter du lait. Ton coloc rentre 10 minutes apres, frigo toujours vide (toi pas encore revenu), il part aussi acheter du lait. Resultat : 4 litres de lait, dont la moitie va tourner. Ce que vous avez fait, en informatique, c'est lire l'etat partage puis prendre une decision basee sur cet etat — sans savoir que l'autre etait en train de faire la meme chose.

Cas concret : Instagram, post le plus like de tous les temps. Des milliers d'utilisateurs cliquent sur le coeur en meme temps. Chaque serveur web execute ce code :

```
rows = db.execute('SELECT likes FROM posts WHERE id = ?', post_id)
current = rows[0]['likes'] # vaut 100
db.execute('UPDATE posts SET likes = ? WHERE id = ?',
           current + 1, post_id)
```

Si *deux serveurs* A et B executent ce code en meme temps : A lit 100, B lit 100 (avant que A ait ecrit), A ecrit 101, B ecrit 101. Tu as perdu un like. Quand tu as 10 000 likes simultanes, tu en perds peut-etre 5 000. C'est une **race condition**.

Solution : transactions

```
BEGIN TRANSACTION;
SELECT likes FROM posts WHERE id = ?;
UPDATE posts SET likes = likes + 1 WHERE id = ?;
COMMIT;

-- En cas de probleme :
ROLLBACK; -- annule tout
```

Une **transaction** est un bloc d'operations qui s'execute soit en entier, soit pas du tout. Pendant la transaction, SQL pose un *verrou* (lock) sur les lignes concernees pour empecher d'autres transactions de les modifier. C'est la version informatique de mettre un cadenas sur le frigo.

DAVID MALAN — lecture

Malan precise : SQLite verrouille la base entiere par default, ce qui ne passe pas a l'echelle. MySQL/Postgres ont un verrouillage plus fin (par ligne) qui permet a milliers de transactions de coexister sans se bloquer. Pour le PSet, tu n'auras pas a manipuler de transactions — mais retiens le mot pour plus tard.

15. Python + SQL avec la librairie CS50

Toute la puissance de SQL devient encore plus interessante quand tu l'embarques dans un programme Python (ou plus tard dans une app web Flask). La librairie `cs50` fournit une couche tres simple :

```
from cs50 import SQL

# Connexion
db = SQL('sqlite:///favorites.db')

# Toute requete s'execute via db.execute(...)
rows = db.execute(
    'SELECT language, COUNT(*) AS n '
    'FROM favorites GROUP BY language ORDER BY n DESC'
)

# rows est une LISTE de DICTIONNAIRES
# [{'language': 'Python', 'n': 190},
#  {'language': 'C',      'n': 58},
#  {'language': 'Scratch', 'n': 24}]

for row in rows:
    print(f"{row['language']}: {row['n']}")
```

Notes importantes :

- `db.execute()` renvoie une liste de dicts pour les SELECT, et un nombre (lignes affectees) pour INSERT/UPDATE/DELETE.
- Les placeholders `?` sont remplaces par les arguments suivants dans l'ordre.
- La syntaxe d'URL `sqlite:///nom.db` avec trois slashes est imposee par la librairie (les deux premiers sont le separateur de schema URI, le troisieme est le chemin).

KELLY — section

En section, Kelly insiste : tu ecris ton SQL comme une **chaîne de caracteres** Python. Ca veut dire que tu peux la decouper sur plusieurs lignes avec concatenation implicite (deux chaines collees). C'est un grand classique du PSet Movies : tes requetes deviennent assez longues, decoupe-les pour qu'elles restent lisibles.

16. Cheat sheet — les requetes a memoriser

Ces 12 patterns couvrent 90% du PSet 7 (Songs + Movies + Fiftyville).

Besoin	Requete
Toutes les colonnes	SELECT * FROM table;
Colonnes choisies	SELECT col1, col2 FROM table;
Filtrer	SELECT * FROM table WHERE col > 10;
Pattern matching	SELECT * FROM songs WHERE name LIKE '%love%';
Trier	SELECT * FROM table ORDER BY col DESC;
Limiter	SELECT * FROM table LIMIT 10;
Compter	SELECT COUNT(*) FROM table;
Moyenne	SELECT AVG(col) FROM table;
Grouper	SELECT col, COUNT(*) FROM t GROUP BY col;
Sans doublons	SELECT DISTINCT col FROM table;
Inserer	INSERT INTO t (a,b) VALUES ('x',1);
Modifier	UPDATE t SET a='y' WHERE id=3;
Supprimer	DELETE FROM t WHERE id=3;
Nested SELECT (=)	SELECT x FROM a WHERE id = (SELECT id FROM b WHERE ...);
Nested SELECT (IN)	SELECT x FROM a WHERE id IN (SELECT id FROM b WHERE ...);
JOIN simple	SELECT a.x, b.y FROM a JOIN b ON a.id = b.a_id;
Index	CREATE INDEX i ON table (col);

Ordre des clauses dans un SELECT (a memoriser) :

```
SELECT    colonnes
FROM      table
JOIN      autre_table ON ...    -- avant WHERE
WHERE     condition             -- filtre les lignes
GROUP BY  colonne              -- regroupe pour agreger
HAVING    condition            -- filtre les groupes (apres GROUP BY)
ORDER BY  colonne ASC|DESC     -- trie le resultat
LIMIT     n;                  -- coupe a n lignes
```

Moyen mnemotechnique : **S**ome **F**rench **J**ournalists **W**rite **G**reat **H**istorical **O**pinions **L**ittered.

17. PSet 7 — ce qui t'attend

Le PSet 7 a trois parties graduees. Voici l'ordre conseille et le pattern principal de chacune.

Songs (warm-up)

8 fichiers `.sql` a remplir. Une seule table, pas de JOIN. Tu pratiques SELECT, WHERE, ORDER BY, LIMIT, AVG, GROUP BY, et un nested SELECT (1 seul). Tu finis ca en une apres-midi si tu as bien revu la fiche.

Movies

12 fichiers. Database avec plusieurs tables (movies, people, stars, directors, ratings, genres). Tu vas faire intensivement des nested SELECT et commencer a toucher au JOIN. Difficulte qui monte progressivement, dernieres requetes exigent de penser le probleme avant d'ecrire la requete.

Fiftyville

Un mini-jeu d'enquete : un crime a ete commis, tu as une base de donnees avec horaires, transcriptions d'interrogatoires, transactions bancaires, retraits ATM, voyages en avion, registres telefoniques. Tu dois identifier le coupable, la ville ou il s'est enfui, et son complice — uniquement avec des requetes SQL. Tres satisfaisant. Compte 2-4h.

CONCEPT CLE

Strategie pour Fiftyville : commence par lire le crime scene report, note tout ce que tu apprends (heure, lieu, indices). Utilise `.schema` pour decouvrir toutes les tables disponibles. Procede par *elimination* : a chaque etape, tu reduis la liste de suspects/destinations/complices possibles. Utilise des nested SELECT en cascade. C'est exactement le raisonnement d'un detective — et tu vas voir, ca rend SQL *fun*.

18. Checklist anti-pieges

Avant d'executer une requete destructive

Reflexe absolu pour UPDATE et DELETE :

```
-- 1. Ecrire d'abord en SELECT pour voir les lignes affectees
SELECT * FROM friend WHERE name = 'Bob';

-- 2. Verifier que c'est bien ce qu'on veut

-- 3. Remplacer SELECT * par DELETE / UPDATE
DELETE FROM friend WHERE name = 'Bob';
```

Erreurs frequentes en debut de PSet

Symptome	Cause probable
Pas de resultat (0 lignes)	Casse des chaines (LIKE est insensible mais = est sensible en general)
Erreur de syntaxe SQL	Apostrophe manquante autour d'une chaine, ou guillemets doubles au lieu de simples
Ambiguous column name	Plusieurs tables ont la meme colonne. Prefixer : table.col
NULL bizarre dans le resultat	JOIN sur une colonne ou une ligne n'a pas de correspondance
Resultat trop gros	Oubli du WHERE ou du LIMIT
Agregation incorrecte	GROUP BY oublie sur une colonne non agregee
Trop lent	Pas d'index sur la colonne du WHERE

Commandes SQLite a connaitre

Commande	Action
.schema	Affiche la structure de toutes les tables
.schema table_name	Schema d'une seule table
.tables	Liste les tables de la base
.mode csv	Active le mode CSV pour les imports
.import file.csv t	Importe file.csv dans la table t
.timer ON	Affiche le temps de chaque requete
.headers ON	Affiche les noms de colonnes en SELECT
.quit ou .exit	Sort de SQLite

19. Bridge vers la suite

Pour CS50. Apres SQL, il te reste essentiellement Web (Flask), Python approfondi, et le projet final. SQL revient massivement dans Web : tu vas construire de petits sites Flask qui lisent et ecrivent dans une base SQLite, exactement comme dans le code d'Instagram simplifie qu'on a vu pour les race conditions.

Pour ta transition AI/ML. SQL est *massivement* utile, plus que ce que les debutants imaginent. En entreprise, 70% du travail d'un ML engineer junior consiste a extraire et nettoyer des donnees depuis une base SQL avant meme de toucher au modele. Tu en feras a Jedha (Phase 4), tu en feras dans tous les projets MS-AI Boulder qui touchent aux donnees tabulaires (ISLP chapitres 3-7), tu en feras dans n'importe quel job remote AI/ML.

Concept SQL avance que tu rencontreras plus tard, mais que CS50 ne couvre pas :

- **LEFT JOIN, RIGHT JOIN, FULL OUTER JOIN** — variantes du JOIN qui gardent les lignes sans correspondance.
- **WINDOW FUNCTIONS** (`OVER (PARTITION BY ...)`) — agregations qui ne reduisent pas le resultat. Tres utilisees en data science.
- **CTE (Common Table Expressions)** avec `WITH` — alternative plus lisible aux nested SELECT profonds.
- **VIEWS** — requetes SELECT sauvegardees comme des tables virtuelles.
- **Pandas + SQL** — `pd.read_sql_query(...)` lit le resultat d'une requete SQL directement dans un DataFrame. Combo gagnant pour la data science.

CONCEPT CLE

Pour la suite immediate : attaque **Songs** aujourd'hui ou demain, en ouvrant cette fiche en parallele pour Ctrl+F. Tu vas voir, c'est tres rapide. Movies te prendra plus longtemps. Fiftyville sera ton moment fun. Quand tu as termine les trois, on fait un point planning — tu seras pile dans les temps pour finir mi-mai et passer a Phase 2 (Maths for ML).

Fin de la fiche — CS50 Semaine 7 SQL. Bonne chance pour le PSet, Flavien.